



Part 1. The Program

In this lab you will create a “Guess the Secret Number” game. The program will generate a secret number between 1 and 100 using the GetRandom library subroutine. It will then repeatedly ask the user to guess the number. After each guess, the program will tell the user whether their guess was too high or too low. When the user finally guesses the correct number, the program will congratulate them and exit.

Important: This lab uses **registers only** to store integer values. You will **not** use memory (no .space buffers) for the secret number or the guess. The .data section is used only for string constants (prompts and messages). All integer values live in registers.

Here is an example of the program running:

```
I am thinking of a number between 1 and 100.  
  
Enter your guess: 50  
Too high!  
  
Enter your guess: 25  
Too low!  
  
Enter your guess: 37  
Too low!  
  
Enter your guess: 42  
You got it! You win!
```

Part 2. Connecting to the Server

If you have not already done so, follow the connection steps from Lab 1 to log in to the server.

- Open MobaXTerm.
- Click Session → SSH.
- Enter the Remote Host:

- Enter your Saclink username and password.

Part 3. The Pseudocode

Below is the complete logic for the program, written in Gaddis pseudocode. Your job is to translate this pseudocode into x86-64 assembly language using Intel syntax and the CSC 35 library.

Read through the entire pseudocode before you start writing assembly. Pay special attention to the control logic: the Do...While loop and the If/Else If structure inside it.

```
Module Program()

    // --- Declare String Constants ---
    Declare String welcome = "I am thinking of a number between 1 and 100.\n"
    Declare String prompt  = "\nEnter your guess: "
    Declare String highMsg  = "Too high!\n"
    Declare String lowMsg   = "Too low!\n"
    Declare String winMsg   = "\nYou got it! You win!\n"

    // --- Generate the Secret Number (1 to 100) ---
    Set secret = Random(100)      // returns 0 to 99
    Set secret = secret + 1       // now 1 to 100

    // --- Welcome Message ---
    Display welcome

    // --- Game Loop ---
    Do
        Display prompt
        Input guess                // read an integer

        If guess > secret Then
            Display highMsg
        Else If guess < secret Then
            Display lowMsg
        End If

        While guess != secret

            // --- Win! ---
            Display winMsg

    End Module
```

Part 4. Creating Your Source Code

Use the Nano text editor to create your file. Type **nano lab3.asm** and press Enter.

```
nano lab3.asm
```

Begin your file with the following structure. You must fill in the .data section with your string constants and write the program logic by translating the pseudocode from Part 3.

```
# -----  
# Lab 3: Guess the Secret Number  
# -----  
.intel_syntax noprefix      # Use Intel formatting  
  
.data                      # The DATA section (string constants)  
  
    # YOUR STRING LABELS GO HERE  
  
.text                      # The TEXT section (code)  
.global Program            # Make the entry point visible  
  
Program:  
  
    # YOUR CODE GOES HERE  
  
    call ProgramEnd        # Terminate the program
```

Press **Control+X**, then **Y**, then **Enter** to save and exit.

Part 5. Assemble the Program

Assemble your source file into an object file. Type **as -o lab3.o lab3.asm** and press Enter.

```
as -o lab3.o lab3.asm
```

If you have any typos, the assembler will list the errors with line numbers. Open the file again with nano and fix them. Common mistakes in this lab include misspelled label names, forgetting the colon after a label (e.g., game_loop: not game_loop), and putting cmp or jump instructions in the wrong order.

Part 6. Link the Objects

Link your object file with the CSC 35 library. Type **ld -o a.out lab3.o csc35.o** and press Enter.

```
ld -o a.out lab3.o csc35.o
```

If you receive “undefined reference” errors, double-check that your label names match exactly (capitalization matters). If you see “undefined reference to Program”, make sure you have `.global Program` in your `.text` section.

Part 7. Execute

Run your program by typing `./a.out` and pressing Enter.

```
./a.out
```

Test your program thoroughly. Try guessing the correct number, guessing too high, guessing too low, and guessing on the boundaries (1 and 100). Make sure the loop continues until you guess correctly and that the program exits after displaying the win message.

Part 8. Submitting Your Lab

Grading Rubric

Category	Points	Description
Assembly & Execution	2	The program assembles (as), links (ld), and executes without errors.
Random Generation	2	The secret number is generated using <code>GetRandom</code> with the correct range (1–100). The result is stored in a register (not memory).
Game Loop	4	The <code>Do...While</code> loop is correctly implemented using labels and jumps. The loop repeats until the guess matches the secret number.
Comparison & Branching	3	The <code>cmp</code> instruction correctly compares the guess to the secret. Conditional jumps (<code>je</code> , <code>jl</code> , <code>jle</code>) route to the correct feedback messages.
Output Messages	2	All pseudocode <code>Display</code> statements are correctly translated. Welcome, prompt, high, low, and win messages all appear at the correct times.
Formatting & Style	2	Code follows standard indentation (instructions indented, labels flush left). Comments are included. File is submitted as <code>.asm</code> .
TOTAL	15	



This activity may only be submitted using the assembly format used in class: (1) x64, (2) Intel-syntax, and (3) the CSC 35 library.

Using AT&T format will result in a zero. Any work from a prior semester will receive a zero.

Select the “.asm” file in the MobaXTerm File Explorer pane and download it to the ECS Lab Computer. Once downloaded, open the Canvas Assignment and submit the “.asm” file.

Useful UNIX Commands

Editing

Action	Command	Notes
Edit File	nano filename	“Nano” is an easy to use text editor.
Assemble File	as -o object source	Don't mix up the object and asm fields. It will destroy your program!
Link File	ld -o exe object(s)	Link and create an executable file from one (or more) object files.

Folder Navigation

Action	Command	Notes
Change current folder	cd foldername	“Changes Directory”
Go to parent folder	cd ..	Think of it as the “back button”.
Show current folder	pwd	Gives the current file path.
List files	ls	Lists the files in current directory.

File Organization

Action	Command	Notes
Create folder	mkdir foldername	Folders are called directories in UNIX.

Copy file	cp oldfile newfile	Make a copy of an existing file.
Move file	mv filename foldername	Moves a file to a destination folder.
Rename file	mv oldname newname	Note: same command as “move”.
Delete file	rm filename	Remove (delete) a file. There is no undo.